Quandela: Review in photonic quantum computing and its applications on graph problems

Eliott Rambeau and Rodrigo Martinez

January 2024

Contents

1 Introduction

Nowadays, there are several ways to encode quantum information into physical systems in order to support quantum computing, such as trapped ions, superconducting circuits, neutral atoms and photons [7]. To do so, we must encode the quantum information in these systems, and be able to manipulate them at our will. This fact makes the platforms more efficient for certain tasks and certainly harder to manage for other ones. In this work we will focus on photons as a quantum information carrier, and photonic circuits as the tool to manipulate it. Indeed, among others, the french start-up Quandela focuses on photonic quantum computing. Their work spans from hardware to theory, algorithms, applications and even quantum machine learning.

Photons present the particular advantage of low coherence, and a wide range of opportunities to encode the information. Throughout the years, several proposals for universal fault-tolerant computing have been put forward in the discrete-variable photonic approach wherein quantum information is encoded with single photons. This encourages the development of a hardware based on integrated photonic chips and single-photon sources and detectors.

Let's break this brief introduction down into three building blocks: sources, manipulation, and detection of single photons.

1.1 Single-photon sources

The scaling of photonic quantum computing requires efficient sources of indistinguishable single photons.

By inserting a semiconductor quantum dot (QD) into a cavity, it is possible to modify its spontaneous emission, and fabricate a ultra-bright single-photon source. Note that, in this context, 'brightness' refers to the probability of emiting that photon. In the absence of a cavity, this brightness is hardly few percent. However, the aimed indistinguishability of the photons here is limited by charge noise [5]. On the other hand, parametric down-conversion sources provide highly indistinguishable photons, but operates at a very low brightness to achieve so.

Placing Quantum dots in electrically controlled caivities is shown to strongly reduce charge noise. N. Somaschi et al.[6] achieved under resonant excitation an indistinguishability of 0.9956 ± 0.0045, demonstrated with a second-order correlation factor of $g^{(2)}(0) = 0.0028 \pm 0.0012$. Moreover, a photon extraction of 65% and a measured brightness of 0.154 ± 0.015 make this source 20 times brighter that any other of equal quality. This enables the french start-up to build near-optimal single-photon sources.

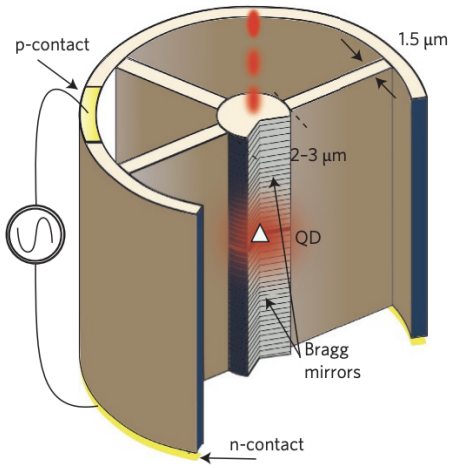

Figure 1: Schematic of the devices under study: a micro-pillar coupled to a QD is connected to a surrounding circular frame by four one-dimensional wires. The top p-contact is defined on a large mesa adjacent to the frame. The n-contact is deposited on the back of the sample.

1.2 Single-photon manipulation

Once we are able to initialise the input single photon states, we want to have a quantum processor in order to perform gates. Since our choice for quantum information carriers are single-photons, we can manipulate them with linear optical circuits. Moreover, note that the quantum states that we are aiming to process, vary depending on the spatial mode on which the photon is traveling; this concept is well known as dual rail encoding. Thus, the gates acting on these quantum states should enable us to manipulate at our will the path of the photons.

So, the current scenario is the following: we want to reconstruct any aimed discrete unitary U by using linear optical devices. How can we do that? We allude to the known property in the field of quantum algorithms [3] which states that any arbitrary n-dimensional

unitary can be approximated to $O(n^2)$ terms by decomposing it into general 2-mode interactions.

Moreover, it is known that we can achieve the most general transformation in $U(2)$ by a variable (lossless) beam splitter, together with a phase shifter [14], transforming the input state with modes (k_1, k_2) into the output state with mode (k_1', k_2'):

$$\begin{bmatrix} k_1' \\ k_2' \end{bmatrix} = \begin{bmatrix} e^{i\phi} sin(w) & e^{i\phi} cos(w) \\ cos(w) & -sin(w) \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} \tag{1}$$

Where the parameter w describes the reflectivity ($\sqrt{R} = sin(w)$) and transmittance ($\sqrt{T} = cos(w)$) of the beam splitter, and the phase ϕ can be obtained by placing a phase shifter after the beam splitter.

Note that we can reconstruct the behavior of a beam splitter with arbitrary reflectivity thanks to a Mach-Zehnder interferometer and 50:50 beam splitters as shown in 2:

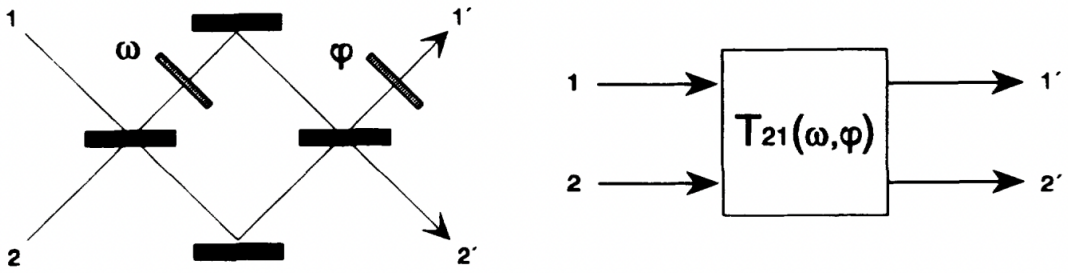

Figure 2: Reconstruction of beam a splitter with arbitrary reflectivity behavior

Thus, the aforementioned properties enables Quandela to implement any arbitrary unitary transformation just by an experimental set up that involves 50:50 beam splitters, Mach-Zehnder interferometers and phase shifters. Note that here we are using path-encoding, since the gates are manipulating the spatial modes of the photons.

1.3 Single-photon detection

The final part required to perform photonic quantum computation is the measurement. To do so, Quandela works with single-photon detectors (SPDs). There exist more than one possible choice for the hardware of SPDs, and Quandela's bet is Superconducting Nanowire Detectors. Let's explain how do they work step by step:

- Superconducting State Transition: Superconducting nanowire detectors exploit the quantum mechanical nature of superconductivity. When a photon is absorbed, it breaks Cooper pairs, leading to a transition from the superconducting to a resistive state.
- Voltage Biasing: The nanowire is biased with a voltage slightly below its critical current. In the superconducting state, the current passes through without resistance. Upon photon absorption, the nanowire briefly becomes resistive, allowing a detectable current to flow.
- Quenching and Resetting: Similar to Avalanche Photo-diodes, superconducting nanowire detectors also require a quenching mechanism to reset the system for subsequent detections. This typically involves applying a magnetic field or adjusting the biasing conditions to restore the superconducting state.

2 Mathematical preliminaries and encoding

2.1 Notation and definitions

In this section, the main necessary definitions will be defined.

Through this report, the state of n single photons in m modes will be written as:

$$|n\rangle = \bigotimes_{i=1}^{m} |n_i\rangle \tag{2}$$

With n_i the number of photons in the ith mode. There are m different possible single photon states and n single photon which means that the number of orthogonal states is:

$$\binom{m+n-1}{n} = \frac{(m+n-1)!}{n!(m-1)!} \tag{3}$$

These states live in a Hilbert space $\mathcal{H}_{m,n}$ which is isomorphic to $\mathbb{C}^M$ [8] where:

$$M = dim_{\mathbb{C}}\mathcal{H}_{m,n} = \binom{m+n-1}{n} \tag{4}$$

Also, note that the set of m×m unitary matrices will be denoted by $U(m)$.

Definition 2.1.1. *Let $U \in U(m)$, $|n_{in}\rangle = |n_{in,1}, ..., n_{in,i}\rangle$ and $|n\rangle = |n_1, ..., n_j\rangle$. Then, one can construct $U_{n_{in},n}$ thanks to the following protocol:*

- *Start by constructing $U_{n_{in}}$ by taking n_i times the ith rows of the U matrix, creating a m*i matrix.*
- *Then take n_j times the jth columns of $U_{n_{in}}$ to construct $U_{n_{in},n}$ a j*i matrix*

Here is an example of how this protocol works:

Example 2.1.2. *Let $U \in U(4)$ be:*

$$U = \begin{pmatrix} 1 & 3 & 5 & 6 \\ 2 & 1 & 9 & 8 \\ 4 & 2 & 8 & 10 \\ 45 & 2 & 7 & 12 \end{pmatrix} \tag{5}$$

And $|n_{in}\rangle = |2, 1, 0, 0\rangle$ and $|n\rangle = |0, 1, 1, 1\rangle$. Then we have:

$$U_{n_{in}} = \begin{pmatrix} 1 & 3 & 5 & 6 \\ 1 & 3 & 5 & 6 \\ 2 & 1 & 9 & 8 \end{pmatrix} \tag{6}$$

And:

$$U_{n_{in},n} = \begin{pmatrix} 3 & 5 & 6 \\ 3 & 5 & 6 \\ 1 & 9 & 8 \end{pmatrix} \tag{7}$$

Thus, we went from a 4×4 matrix to a 3×3 matrix. Note that this is very useful since even if the states live in a 4×4 Hilbert space, there are only 3 photons. This means that the matrix we need to implement has to be 3×3. Moreover, this allows to take into account how many photons are in each modes, and to create a 'weighted' matrix. Which conforms the 'effective' transformation acting on the non-empty inputs.

We now define few useful mathematical concepts.

Definition 2.1.3. *The **permanent** of a matrix A is defined by:*

$$Per(A) = \sum_{\sigma \in S_n} \prod_{i}^{n} a_{i\sigma(i)} \tag{8}$$

With a_{ij} the coefficients of the matrix and S_n the symmetric group.

Definition 2.1.4. *The **adjacency matrix** of a simple graph with vertex set $U = \{u_1, \ldots, u_n\}$ is defined as a square n*n matrix A such that its element A_{ij} are 1 when there is an edge from vertex u_i to vertex u_j, and 0 when there is no edge. The diagonal elements of the matrix are all zero, since edges from a vertex to itself (loops) are not allowed in simple graphs.*

Definition 2.1.5. *The **spectral norm** of a matrix A id defined by:*

$$\|A\|_s = max\{\|Ax\|_2 : \|x\| = 1\} \tag{9}$$

Definition 2.1.6. *A **permutation matrix** is a square binary matrix that has exactly one entry of elements equal to 1 in each row and each column with all other elements equals to 0.*

Example 2.1.7. *For example, the following matrix P perform the following permutations represented by π:*

$$P = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} \Longleftrightarrow \pi : \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 2 & 4 & 1 \end{pmatrix} \tag{10}$$

In order to permute the rows of a matrix M, one has to compute PM, and to permute the columns, one has to compute MP^T.

Definition 2.1.8. *(Graph isomorphism). Suppose two directed or undirected graphs G_1 and G_2 with adjacency matrices A_1 and A_2 are given. G_1 and G_2 are **isomorphic** if and only if there exists a matrix P such that:*

$$PA_1P^T = A_2 \tag{11}$$

2.2 Mathematical preliminaries

In this part, the mathematical concepts and theorems needed to implement the desired algorithm will be introduced.

Theorem 2.2.1. *Let U be a unitary matrix acting on a 1 photon, m modes system. There exists a natural homomorphism $\psi : m*m \to n*n_{in}$ which maps this unitary to the corresponding one, acting on n photons. Then, one has:*

$$Per(U_{n_{in},n}) = \langle n| \psi(U) |n_{in}\rangle \sqrt{\prod_i n_{in,i}! \prod_j n_j!} \tag{12}$$

Proof:
See [1]. □

Lemma 2.2.2. *Let $U \in U(m)$ be the unitary matrix acting on one qubit we want to implement. Let ψ be the homomorphism defined as above and $\mathcal{U} = \psi(U) = U_{in,n}$ be such that:*

$$\psi(U) |n_{in}\rangle \coloneqq \mathcal{U} |n_{in}\rangle = \sum_n \gamma_n |n\rangle \tag{13}$$

Then, the conditional probability is:

$$p(n|n_{in}) \coloneqq p_U(n|n_{in}) = |\gamma_n|^2 = \frac{|Per(U_{in,n})|^2}{\prod_i n_{i,in}! \prod_j n_j!} \tag{14}$$

Proof:
By definition:

$$p_U(n|n_{in}) = |\langle n| \mathcal{U} |n_{in}\rangle|^2 \tag{15}$$

$$p_U(n|n_{in}) = |\langle n| \psi(U) |n_{in}\rangle|^2 \tag{16}$$

$$\Longleftrightarrow \; p_U(n|n_{in}) = |\frac{Per(U_{n_{in},n})}{\sqrt{\prod_i n_{in,i}! \prod_j n_j!}}|^2 \tag{17}$$

$$\Longleftrightarrow \; p_U(n|n_{in}) = \frac{|Per(U_{n_{in},n})|^2}{\prod_i n_{in,i}! \prod_j n_j!} \tag{18}$$

□

This Lemma is very important since it tells us that by studying the statistics of an experiment, one can get the permanent of the unitary matrix. And, as we will see later, the permanent of a graph can be used to solve many graphs problems.

2.3 Block Encoding of A

In the previous section, it has been shown how to get the permanent of an unitary matrix thanks to the statistics of the measurement. This report takes interest in graph theory and graphs problems. However, the adjacency matrix of a grap is in general not unitary. This means that we can't apply it directly to a quantum state, i.e it can't be directly implemented in a quantum circuit. The following section will provide a technique to overcome this limitation. Let's start by a definition:

Definition 2.3.1. *Let A be the matrix of interest, and $\sigma_i(A)$ its set of singular value. Then, a **scaled matrix** is defined by:*

$$A_s \coloneqq \frac{1}{\sigma_{max}} A \tag{19}$$

Which leads to $\sigma_{max}(A_s) \leq 1$

The following theorem is the one used to encode non unitary matrix in quantum system.

Theorem 2.3.2. *(Unitary dilatation theorem[17]). Let A be a non unitary matrix such that $\|A_s\|_s \leq 1$, then A can be embedded in a larger unitary matrix, technique called* ***block encoding****:*

$$\begin{pmatrix} A & \sqrt{\mathbb{I}_{n*n} - AA^\dagger} \\ \sqrt{\mathbb{I}_{n*n} - A^\dagger A} & -A^\dagger \end{pmatrix} \tag{20}$$

Proof We want to prove that one can embeds a non unitary matrix A, which lives in a Hilbert space $\mathscr{H}$ and has $\|A\|_s \leq 1$, into a unitary matrix U, which lives in a larger Hilbert space $\mathscr{H}'$. That is to say, we want to show that there exists some projector P and a unitary matrix U such that:

$$P_{\mathscr{H}} U|_{\mathscr{H}'} = A_{\mathscr{H}} \tag{21}$$

If P project on the subspace represented by the 0th elements of the U matrix, then any unitary matrix having A in its 0th elements would be good. However, there are two limitations:

- It needs to be unitary.
- It needs to be a dilatation, i.e $U \in \mathscr{H}' = \mathscr{H} \oplus \mathscr{H}$

To complete those criterion, one construct explicitly a matrix and prove it is unitary. The matrix is:

$$U = \begin{pmatrix} A & \sqrt{\mathbb{I}_{n*n} - AA^\dagger} \\ \sqrt{\mathbb{I}_{n*n} - A^\dagger A} & -A^\dagger \end{pmatrix} \tag{22}$$

The fact that $U \in \mathscr{H} \oplus \mathscr{H}$ is quite obvious. Moreover, for this matrix to exist, one needs:

$$\mathbb{I}_{n*n} - AA^\dagger \geq 0 \iff \|A\|_s \leq 1 \tag{23}$$

Which is the case by the assumptions of the theorem. As it has been said, this matrix lives in $\mathscr{H} \oplus \mathscr{H}$ and we have:

$$UU^\dagger = \begin{pmatrix} AA^+\sqrt{\mathbb{I}_{n*n} - AA^\dagger}(\sqrt{\mathbb{I}_{n*n} - AA^\dagger})^\dagger & A(\sqrt{\mathbb{I}_{n*n} - A^\dagger A})^\dagger - \sqrt{\mathbb{I}_{n*n} - AA^\dagger}A \\ \sqrt{\mathbb{I}_{n*n} - A^\dagger A}A^\dagger - A(\sqrt{\mathbb{I}_{n*n} - AA^\dagger})^\dagger & \sqrt{\mathbb{I}_{n*n} - A^\dagger A}(\sqrt{\mathbb{I}_{n*n} - A^\dagger A})^\dagger + A^\dagger A \end{pmatrix} \tag{24}$$

Note that $(\sqrt{\mathbb{I}_{n*n} - AA^\dagger})^\dagger = \sqrt{\mathbb{I}_{n*n} - AA^\dagger}$ and $\sqrt{\mathbb{I}_{n*n} - A^\dagger A}^\dagger = \sqrt{\mathbb{I}_{n*n} - A^\dagger A}$. This leads to:

$$UU^\dagger = \begin{pmatrix} \mathscr{I}_n & 0 \\ 0 & \mathscr{I}_n \end{pmatrix} \tag{25}$$

Which proves the theorem. □

To see how this theorem apply to the case of scaled adjacency matrices, one needs to derive a lemma:

Lemma 2.3.3. *Let A be a matrix with singular values $\sigma_i(A)$, then one has:*

$$\|A\|_s = \sigma_{max}(A) \tag{26}$$

Proof:

Let $x = (a_1, ..., a_n)$ be a vector of norm 1 in a vector space which has $\{e_i\}_{i\in[1,n]}$ as basis vector. Let also λ_i be the set of singular values of A^*A. We have:

$$\|Ax\| = \sqrt{< Ax, Ax >} = \sqrt{< x, A^*Ax}} \tag{27}$$

$$\iff \|Ax\| = \sqrt{< \sum_i a_i e_i, \sum_i \lambda_i a_i e_i >} \tag{28}$$

$$\iff \|Ax\| = \sqrt{\sum_i a_i \bar{a_i} \lambda_i} \tag{29}$$

$$\iff \|Ax\| \leq \sqrt{\lambda_{max}}\|x\| = \sqrt{\lambda_{max}} \tag{30}$$

Then, we have:

$$\|A\|_s = max\{\|Ax\|_2 : \|x\| = 1\} \Rightarrow \|A\|_s \geq \|Ax\|_2 \tag{31}$$

$$\|A\|_s^2 \geq < x, A^*Ax > \tag{32}$$

Taking $x = e_j$ with j the columns where the singular values of A is maximum, we get:

$$\|A\|_s^2 \geq < e_j, \lambda_{max} e_j >= \lambda_{max} \tag{33}$$

Combining the eq.30 and eq.33, one gets that $\|A\|_s = \lambda_{max}$ where λ_{max} is the maximum singular value of A. And so, we get the desired results. □

In our case, we have $\sigma_{max}(A_s) \leq 1$ which means that $\|A_s\|_s \leq 1$ and allows us to use the theorem2.3.2 and embed the matrix A_s in a larger unitary $U \in U(2n)$. Then, there exists a 2n circuit to implement this unitary [15]. Note that in order to implement A, one has to find the singular value decomposition of A, which can be done in $O(n^3)$[4]. Moreover, finding the circuit to implement U can be done in $O(n^2)$, which makes this implementation efficient. Finally, note that one can take every $s \geq \sigma(A)_{max}$ to normalize A. However, since $Per(A_s) = \frac{1}{s^n} Per(A)$, one has to take the smallest s to increase probability of having the permanent as an outcome.
As we will see latter, the permanent of the adjacency matrix of a graph is useful to solve many problems, which makes this implementation very useful. Even more if we take:

$$\begin{cases} n_i = n_{in,i} = 1 \text{ if } i \leq n \\ n_i = n_{i,in} = 0 \text{ otherwise} \end{cases} \tag{34}$$

since it gives $n_i! = 1 \forall i$ and so:

$$p(n|n_{in}) = |Per(A_s)|^2 = \frac{1}{s^{2n}} |Per(A)|^2 \tag{35}$$

Note that since the interest is not on the number of photons, one can use threshold detectors to implement this which are relatively simple practically.

3 Applications

In the prior section, we saw that we can implement a desired matrix A (its scaled version A_s) by performing a linear optical circuit to a larger register. Since this matrix has no constraints, we can choose it to be the adjacency matrix (2.1.4) of certain graphs. But, why do we have particular interest in that choice?

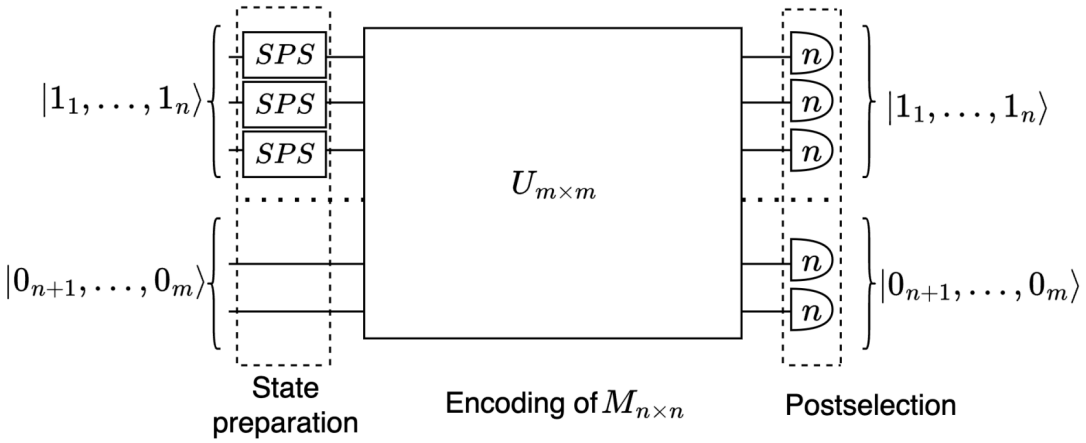

Figure 3: Setup to compute |Per(A)|. Firstly, we initialise the state, composed by n single photons emitted by n single-photon sources and make it pass through a linear optical circuit of size m = 2n encoding A. Afterwards, we detect the output state by placing m single-photon detectors at the output modes.

It is well known that the permanent of the adjacency matrix of a graph can be used to several applications concerning graph problems. Moreover, in 35 we saw the relation between the output distribution and the permanent of A_s. Therefore, we can exploit the setting shown in 3 in order to calculate the permanent and by extension the solution to some graph problems.

3.1 Number of perfect matchings

First of all, let's define what a 'perfect matching' is:

Definition 3.1.1. *(Perfect matching) In graph theory, a perfect matching in a graph is a matching that covers every vertex of the graph. More formally, given a graph G = (V, E), a perfect matching in G is a subset E_M of edge set E, such that every vertex in the vertex set V is adjacent to exactly one edge in E_M.*

Moreover, we know that if G is a bipartite graph where each part's size is equal ($|V_1| = |V_2| = \frac{n}{2}$), then, the number of perfect matchings is given by:

$$pm(G) = \sqrt{Per(A)} \tag{36}$$

Where $pm(G)$ stands for the number of perfect matching of the graph G.
Hence, finding the permanent of the adjacency of this kind of graph allows us to acquire the number of perfect matchings in a graph.

3.2 Permanental polynomials

The technique presented in the paper allows us to find the permanental polynomial of a graph. It is defined by:

$$P_A(x) = Per(x\mathbb{I}_n - A) = \sum_i c_i x_i \tag{37}$$

The goal is to find all the coefficient c_i. To do that, one has to implement the matrix $B_x \coloneqq (x\mathbb{I}_n - A)$ n+1 times with randomly chosen values of x in order to be able to solve the system of equation:

$$\begin{pmatrix} 1 & x_1 & ... & x_1^n \\ ... & & & \\ 1 & x_n & ... & x_n^n \end{pmatrix} \begin{pmatrix} c_0 \\ ... \\ c_n \end{pmatrix} = \begin{pmatrix} P_A(x_1) \\ ... \\ P_A(x_{n+1}) \end{pmatrix} \tag{38}$$

To solve this system the matrix with the x_n should be invertible, i.e have non zero determinant, which is true in the general case.

3.3 Densest subgraph identification

3.3.1 Theory

In this section, we will discuss the resolution of the k-densest subgraph problem, using block encoding and the permanent of the adjacency matrix. The k-densest problem is the problem of finding for a n graph a $k < n$ subgraph with the highest density. Let's define the density:

Definition 3.3.1. *Let G=(V,E) be a graph, then its density is:*

$$D = \frac{|V|}{|E|} \tag{39}$$

Intuitively, one can see that the permanent is linked with the density. Indeed, if the number of vertices increase, the density will too and there will be more terms in the adjacency matrix which cannot make the permanent decrease. We provide here a lemma and its proof, which were not made initially in the paper, some numerical insight were given but no rigorous proof.

Lemma 3.3.2. *Let G be a n graph, A its adjacency matrix and D its density. Then, the permanent of the adjacency matrix is a non-decreasing function of the density of the graph.*

Proof
We want to prove this theorem by induction. We set $n = 2$, then the matrix can be:

$$A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \Rightarrow Per(A) = 0 \tag{40}$$

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \Rightarrow Per(A) = 1 \tag{41}$$

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow Per(A) = 1 \tag{42}$$

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \Rightarrow Per(A) = 1 \tag{43}$$

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow Per(A) = 2 \tag{44}$$

In the first case the density is 0, in the second it is $\frac{|V|}{|E|} = \frac{1}{2}$, in the third and fourth case D=1 and in the last one it is $\frac{3}{2}$. This prove the case n=2.

Then, we suppose that the lemma is true for n and want to prove it for n+1. We define a matrix A_{n+1} of a graph of size $(n+1)$. Then, we know that:

$$per(A_{n+1}) = \sum_i a_i Per(A_{ij}) \tag{45}$$

With A_{ij} the matrix A_{n+1} without the ith row and the jth column. All those matrices are n*n matrices and so, by assumption, their permanent increase with the density. Moreover, note that it is a sum of increasing positive terms (the permanents) with a_{ij} as coefficients. When all the coefficients are 0, then the density is 0. Then, if the density increase, a coefficient of the matrix will become 1. This coefficient can be one of the sum's coefficient or not. Either case, the permanent can't decrease with the density and the lemma is proven. □

Thanks to this lemma, one can say that searching the biggest permanent of a sub-graph of a graph is equivalent to solving the k-densest graph problem. Thanks to lemma 2.2.2, we see that the biggest the permanent is, the more chances it has to be measured, which is good for our study. We may find few subgraphs with the same permanent but then, it suffices to check those graphs by hand to solve the problem.
Practically, one can encode the adjacency matrices of all the subgraphs in a matrix K:

$$K = \begin{pmatrix} A_{n_1,n_1} & 0 \\ ... & ... \\ A_{n_j,n_j} & 0 \end{pmatrix} \tag{46}$$

After that, one can block encode this matrix in a unitary U_K, which can be implemented thanks to a linear optical circuit, and samples the results. Let denote by $m \in [1, j]$ the index of the densest subgraph, then most frequent outcome is $|n_{out,m}\rangle$ with a probability of:

$$p(n_{out,m}|n_{in}) = \frac{1}{\sigma_{max}^{n+j}} |Per(K_{n_{in},n_{out,m}})|^2 = \frac{1}{\sigma_{max}^{n+j}} |Per(A_{n_m,n_m})|^2 \tag{47}$$

This allows us the find the densest subgraph. However, it requires to implement all possible subgraph, that is $\binom{n}{k}$. This requires an exponential implementation and is not very practical.
We will now show how this quantum algorithm can be used with $O(Poly(n))$ time complexity to complement a classical algorithm [12]. When used alone, this classical algorithm identify in a first time the *exact* first $[\rho k]$ vertices, with $0 \leq \rho \leq 1$, and then choose randomly the last ones. This algorithm runs in an exponential time. The idea presented in the paper [11] is to exactly solve the ρk-densest problem thanks to the classical algorithm and use the quantum one to find the last vertices. In this case, one would need to encode

$$J = \binom{n - \rho k}{(1-\rho)k} \leq n^{(1-\rho)k} \tag{48}$$

matrices in the matrix K. If one chooses:

$$\rho = 1 - O(\frac{1}{k}) \tag{49}$$

Which leads to $J \approx n^{O(1)} = Poly(n)$. Thus, by combining the classical and quantum algorithm, one can solve the k-problem densest.

The above technique was proposed in the paper[11]. However, note that there are some limitations. While the quantum algorithm has to run less time, the classical algorithm has to run more with an exponential time complexity. This means that we have just moved the problem and that the quantum algorithm does not provide a real benefit. Moreover, if one choose ρ to be like in equation49, the classical algorithm will solve the $\rho k = k - O(1)$ problem. Therefore, the quantum algorithm is not really useful anymore.

3.3.2 Implementation

In Quandela's paper[11], they provide a link to a github with all their codes. We learned how to use and implement them. We now apply this to solve the k-densest problem. We first choose the following graph:

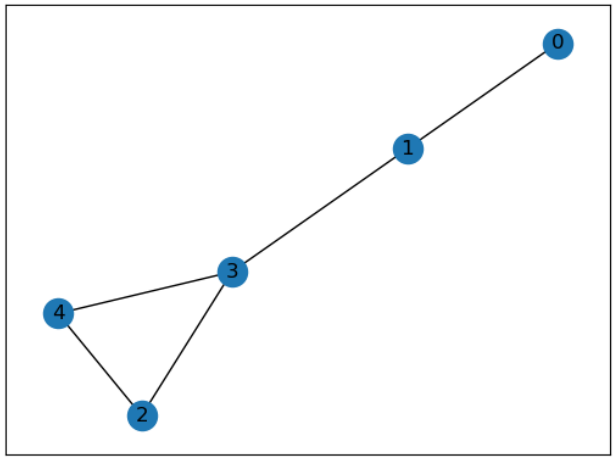

Figure 4: 5 graph.

We want to solve the 3 densest problem in this graph, here are the output results of the program:

```
[(100.0, [2, 3, 4]), (0.0, [1
0, [0, 2, 3]), (0.0, [0, 1, 4
(100.0, [2, 3, 4])
```

Figure 5: Output results for the 3-densest graph problem.

One can see that the graph 2-3-4 is the only one going out, this is indeed the densest subgraph. We have tried the program with several graphs and it always gave the good result.

After doing that, we wanted to study the time complexity of the algorithm. To do that we added few lines to the original code. The idea was to increase the size of the graph and ask Quandela's program to solve the $int(\frac{n}{2}) + 1$-densest subgraph problem, with n the size of the graph. We get the following result:

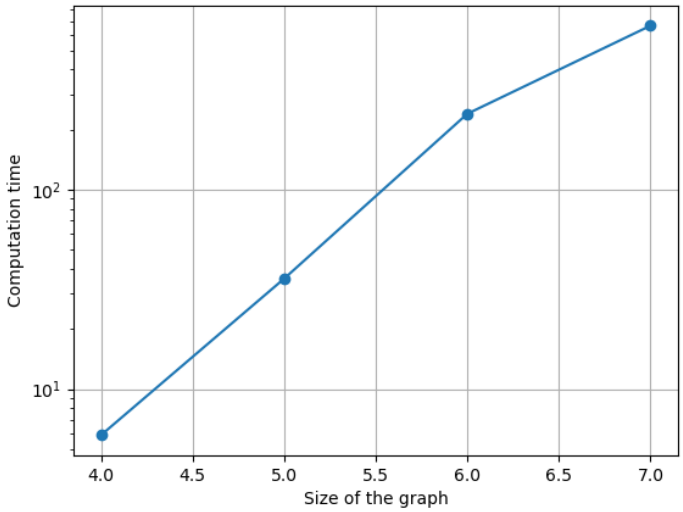

Figure 6: Time complexity of the k-densest subgraph algorithm.

The time dependency is exponential, which was expected from the theory. Those points are all averaged over 5 values.

After that, we wanted to study the impact of solving the ρk-densest problem with a classical algorithm on the quantum time complexity. In order to do that, one start by solving the k-densest problem thanks to the algorithm. Then, one gives the first good vertex of the k-densest subgraph to the algorithm and solve the problem with this new insight. After that, one gives two vertices to the algorithm and solve the remaining problem, and so on. We did that for a 5 vertices graphs, while asking to solve the 3-densest problem. Here are the results:

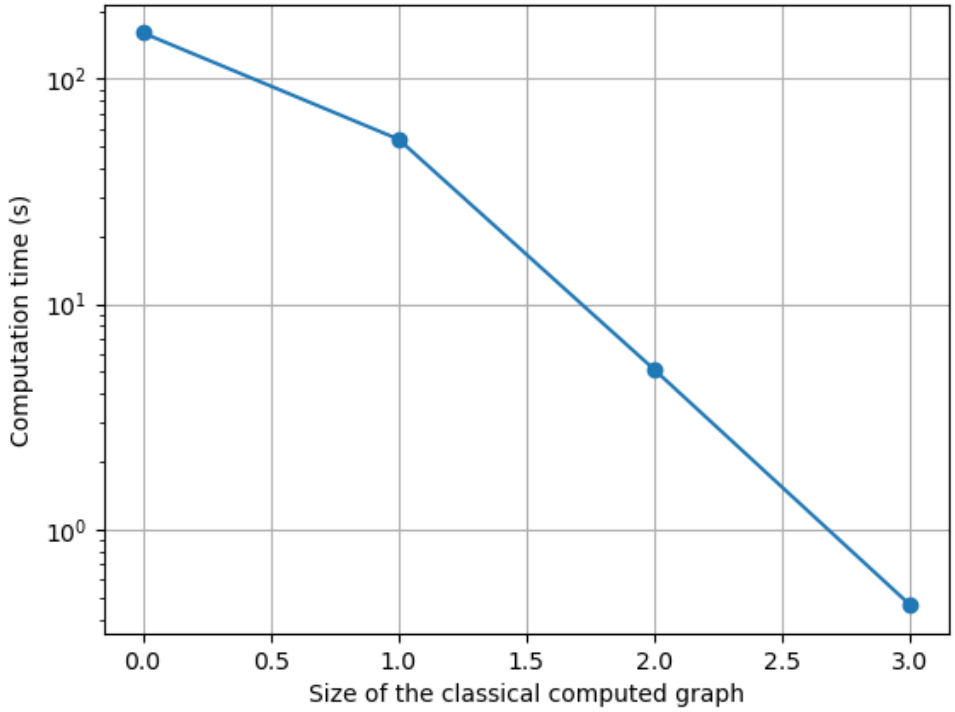

Figure 7: Classical algorithm impact on time complexity.

Once again, the time dependency decrease exponentially, as expected. However note that, while the time dependency goes down exponentially for the quantum algorithm, it increase exponentially for the classical algorithm.

Here is a link to a github containing our code: [10]. Our small contribution to the code is to:

- Create a loop to increase the size of the graph and solve the $int(\frac{n}{2}) + 1$-densest subgraph problem.
- Add a timer in this loop, while making sure that only the running time of the algorithm is taken into account (and not other task).
- Plot the graph.
- Solving the densest problem with classical insight was a feature provided by Quandela. We only had to add timers and plot the graph.

3.4 Graph Isomorphism problem

In the introduction, we defined what it means that two graphs G_1 and G_2 are isomorphic 2.1.8. Now, we are going to see how the computation of the permanent of different adjacency matrices can allows us to determine whether if two graphs are isomorphic or not. This dilemma is denoted as the **graph isomorphism problem**, and at the moment, it is known that it belongs to the the low hierarchy of class NP. The way we can solve these problems using our photonic setup is the following:

Let $l \in \{1, ..., n\}$, $\mathbf{t} = \{t_1, ..., t_l\}$, and $\mathbf{s} = \{s_1, ..., s_l\}$ with $s_i, t_i \in \{1, ..., n\}$, and $s_{i+1} \geq s_i, t_{i+1} \geq t_i \forall i$. Let $B_{\mathbf{t},\mathbf{s}}$ be an l×l sub-matrix of B constructed first by creating an l×n matrix B_s such that the ith row of B_s is the s_ith row of B, and then constructing $B_{\mathbf{t},\mathbf{s}}$ such that its jth column is the t_jth column of $B_{\mathbf{s}}$ 2.1.1.

Theorem 3.4.1. *Let G_1 and G_2 be two unweighted, undirected, isospectral (having the same eigenvalues) graphs with n vertices, and no self loops. Let A and B be the respective adjacency matrices of G_1 and G_2. Then, the two following statements are equivalent:*

1) There exists a fixed bijection $\pi : \{1, ..., n\} \to \{1, ..., n\}$ such that for all $\boldsymbol{s}$, $\boldsymbol{t}$, the following is satisfied:

$$Per(A_{\pi(\boldsymbol{t}),\pi(\boldsymbol{s})}) = Per(B_{\boldsymbol{t},\boldsymbol{s}}) \tag{50}$$

Where $\pi(\boldsymbol{s}) = \{\pi(s_1), ..., \pi(s_l)\}$, $\pi(\boldsymbol{t}) = \{\pi(t_1), ..., \pi(t_l)\}$.

2) G_1 is isomorphic to G_2

Note that this theorem implies that we can solve the Graph Isomorphism problem by designing a protocol which consists in encoding the adjacency matrices of the desired graphs 'A' and 'B' into linear optical circuits U_A and U_B. Then, one can sample the output probability distributions that correspond to the input state with l single-photons (with l taking all possible values from 1 to n), in all possible $\binom{n+l-1}{l}$ arrangements in the first n modes.

Nonetheless, this method is clearly limited by the number of required experiments $\sum_{l=1,...,n} \binom{n+l-1}{l}$, which scales exponentially with n. Thus, it seems convenient to face this problem with 'relaxed' conditions, in order to be able to obtain information in a reasonable amount of time. This relaxed condition is going to be the verification of just a necessary (and not sufficient) condition for the two graphs to be isomorphic. It means that we cannot verify if the graphs are isomorphic, but just the negation.

To do so, we can allude to a particular graph theory property, which states that the equality of the Laplacian permanental polynomials of G_1 and G_2 is a necessary condition for these graphs to be isomorphic [16].

$$P_L(x) = Per(xI_{nxn} - L(G)) \tag{51}$$

Where $L(G)$ is the Laplacian of the graph.

Consequently, as mentioned before for the application of computing permanental polynomials, we can obtain the coefficients of the Laplacian permanental polynomials of G_1 and G_2 with n + 1 experiments (varying the value of x_j for j from 1 to n+1) each and then solving the corresponding system of n + 1 linear equations. To do so we use the fact that we can encode the matrices $x_j I_{nxn} - L(G_1)$ and $x_j I_{nxn} - L(G_2)$ with a linear optical circuit just as we did it for the adjacency matrices. Then, with our photonic setting 3, we sample for each experiment the output probability which gives us an approximation of $P_L(x_j)$

By obtaining these coefficients and comparing them for both graphs we will be able to determine if G_1 and G_2 are **not** isomorphic. Note that if the polynomials are equals, it does not imply that the graphs are isomorphic because it is not a sufficient condition.

4 Sample complexities

During the whole work, we have been assuming that we are able to reconstruct the probability distribution using the output distribution. However, this is not a trivial assumption at all. In fact, it is one of the main limitations when it comes to obtain a quantum-over-classical advantage.

That is, we can only **estimate** $p(\mathbf{n}|\mathbf{n}_{in})$ (and consequently $|Per(A)|$), using the samples obtained from running our experiment many times. We can use the Hoeffding's inequality to see that the estimation can be computed up to an additive error of $\frac{1}{\kappa}$ by performing $O(\kappa^2)$ runs. At this point we immediately see that estimating permanents with this photonic settings present no advantage over classical processing. For example the Gurvits algorithm can estimate permanents within $\frac{1}{poly(n)}$ additive error in $Poly(n)$-time.

Nonetheless, studying the properties of this settings is still of interest. In fact, there are some promising proposals [9] that presents how these noisy intermediate-scale quantum (NISQ) devices could lead in an near term to a practical quantum advantage. This could apply to fields related to quantum chemistry [7], having a direct impact on chemicals industry, drug development, and battery materials research.

5 Probability boosting

In order to perform probability boosting, one needs to increase the permanent of A (2.2.2). To do that, one can notice that if one row of the matrix A is multiplied by $w > 1$, then the permanent will increase and become $wPer(A)$. However, to use the theorem 2.3.2, one needs that the largest singular value of A remains smaller or equal to one. This problem is highly not trivial. In fact only a *necessary* but not *sufficient* condition have been found so that $p_w(n|n_{in}) > p(n|n_{in})$. This condition is given in the following lemma:

Lemma 5.0.1. *Let A be a matrix, and A_w be the same matrix with one of its rows multiplied by $w > 1$. Then:*

$$p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow \frac{\sigma_{max}(A)}{\sigma_{max}(A_w)} > \frac{1}{w^{\frac{1}{n}}} \tag{52}$$

Proof
We have:

$$p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow |Per(A_{s,w})|^2 > |Per(A_s)|^2 \tag{53}$$

Where $A_s, A_{s,w}$ are the scaled matrices and $s = \sigma_{max}$ is the normalization factor.2.3.2. Note that we have:

$$Per(A_s) = Per(\frac{A}{\sigma_{max}}) = \frac{1}{\sigma_{max}} Per(A) \tag{54}$$

Then, one has

$$p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow \frac{|Per(A_w)|^2}{\sigma^{2n}_{max}(A_w)} > \frac{|Per(A)|^2}{\sigma^{2n}_{max}(A)} \tag{55}$$

$$\iff p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow w^2 \frac{|Per(A)|^2}{\sigma^{2n}_{max}(A_w)} > \frac{|Per(A)|^2}{\sigma^{2n}_{max}(A)} \tag{56}$$

$$\iff p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow \frac{\sigma^{2n}_{max}(A)}{\sigma^{2n}_{max}(A_w)} > \frac{1}{w^2} \tag{57}$$

$$\iff p_w(n|n_{in}) > p(n|n_{in}) \Rightarrow \frac{\sigma_{max}(A)}{\sigma_{max}(A_w)} > \frac{1}{w^{\frac{1}{n}}} \tag{58}$$

□

This lemma provide a guideline to find matrices on which we can perform probability boosting. To have a better guideline, one can show that ([11]):

$$\begin{cases} w\gamma < \|A\|_\infty \\ (w^2 - 1)\delta << \sum_i \sigma_i^2(A) \end{cases} \implies \sigma_{max}(A) \approx \sigma_{max}(A_w) \implies \frac{\sigma_{max}(A)}{\sigma_{max}(A_w)} > \frac{1}{w^{\frac{1}{n}}} \tag{59}$$

With:

$$\begin{aligned} \gamma &\coloneqq \sum_j |a_{cj}| \\ \delta &\coloneqq \sum_j a_{cj}^2 \end{aligned} \tag{60}$$

Where c is the number of the line multiplied by w. This condition are very mathematical and due to the shortness of this report, we won't go into details.

The effect of probability boosting will now be numerically shown. Let's take:

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{61}$$

And multiply its 10th row by $w > 1$. Then, the following plot show the evolution of:

$$R \coloneqq \frac{p_w(n_{in}|n)}{p(n_{in}|n} \tag{62}$$

With a w dependency:

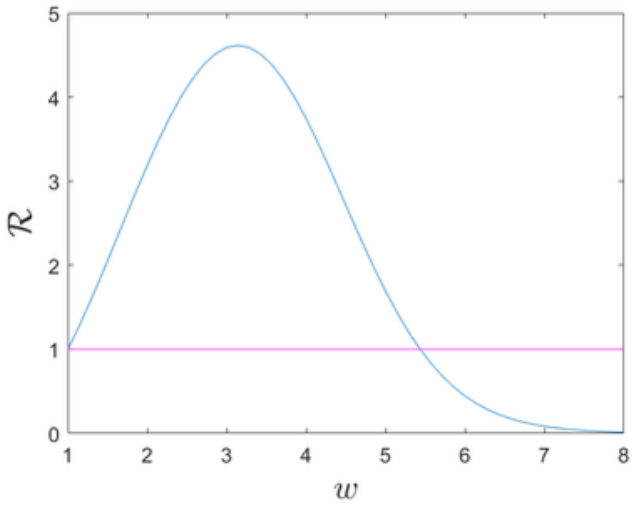

Figure 8: Evolution of R with respect to w.

One can see that the best value of w is 3.2, and in this case the probability is multiplied by 4.5. This is a really good improvement, however note that if one increase w to much, then the previous lemma will be violated and the probability will not increase anymore. Thus, one need to tune very precisely the value of w.

6 Beyond the paper

6.1 Diagonal block encoding

In this section we will propose an extension to Quandela's work. We believe that it is a way to get rid of the limitation on the largest singular value and so to be able to perform more efficient probability boosting. In the paper[13] a technique is provided to perform diagonal block encoding. This technique has no limitation on the singular value. However, it can only be used to encode diagonal matrices; we will provide a way to get over this limitation in the next section. We first start by introducing their technique to block encode a diagonal matrix, thanks to the following theorem:

Theorem 6.1.1. *Given an n-qubit quantum state specified by a state-preparation-unitary U, such that* $|\psi\rangle_n = U|0\rangle_n = \sum_{j=0}^{N-1} \psi_j |j\rangle_n$ *(with* $\psi_j \in \mathbb{C}$*), we can prepare a (1,n+3,0)-block-encoding* U_A *of the diagonal matrix* $A = diag(\psi_0, ..., \psi_{N-1})$ *with* $O(n)$ *circuit depth and a total of* $O(1)$ *queries to a controlled-U gate.*

Proof: See the first appendix 7.

This proof is also provided in the article [13], but is split between many lemmas. We have grouped everything in one proof and kept only the useful parts.

6.2 Proposed modification for adjacency matrices

Last part provide a way to block encode diagonal matrices without any constraints of the singular values. This techniques have a lot of applications[13]. However, our current concern is on graph problems. One needs to find a way to encode non diagonal matrices. Here are the proposed changes to the theorem refdiagonal-block-encoding-theorem:

- Instead of asking for a matrix U such that $U|0\rangle_n = \sum_{j=0}^{N-1} \psi_j |j\rangle_n$, ask for it to be such that:

$$U|0\rangle\langle 0|_n U^\dagger = \sum_{j\neq i}^{N-1} \frac{A_{ij}}{2} |i\rangle\langle j|_n + \sum_{i}^{N-1} (\frac{A_{ii}}{2} |i\rangle\langle i|_n + \frac{1}{2}\mathbb{I}_n) \tag{63}$$

  This new matrix is n×n and can be implemented using quantum circuit. Finding this circuit can be done in $O(n^2)$ [13].

- Define the new total unitary matrix to be (for the definition of $U^{(p)}$, see 82):

$$U_{tot} = U^{(p)}(2|0\rangle\langle 0|_n \otimes \mathbb{I}_{n+2} - \mathbb{I}_{2n+2})U^{\dagger(p)} \tag{64}$$

- Then, the new condition to be satisfied is:

$$\|A - (\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U_{tot} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n)\| = 0 \tag{65}$$

With A the adjacency matrix we want to encode.

Proof:
We provide here a sketch of the proof to show that the last equation holds. It is done by making parallel to the formal proof of 6.1.1, done in appendix 7. This equation is:

$$\|A^{(p)} - (\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U^{(p)} (2 |0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n - \mathbb{I}_{2n+2}) U^{\dagger(p)} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n)\| = 0 \tag{66}$$

Thus, one first needs to compute:

$$U^{(p)} (2 |0\rangle \langle 0|_n \otimes \mathbb{I}_{n+2} - \mathbb{I}_{2n+2}) U^{\dagger(p)} \tag{67}$$

$$\iff 2U^{(p)} (|0\rangle \langle 0|_n \otimes \mathbb{I}_{n+2}) U^{\dagger(p)} - \mathbb{I}_{2n+2} \tag{68}$$

Taking a look at the proof of the theorem(6.1.1), one see that $(\mathbb{I}_n \otimes \langle 0|_{n+2}) U^{(p)} (|0\rangle_{n+2} |k\rangle_n)$ reduces to:

$$\frac{-1}{2} (\mathbb{I}_n \otimes \langle 0|_{n+1}) W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle_{n+1} |k\rangle_n \tag{69}$$

This calculation is made without calling the unitary U and is so exactly the same in our case. We now have to compute:

$$(\mathbb{I}_n \otimes \langle 0|_{n+2}) 2U^{(p)} (|0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n) U^{\dagger(p)} (\mathbb{I}_n \otimes |0\rangle_{n+2}) = -(\mathbb{I}_n \otimes \langle 0|_{n+1}) W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle \langle 0|_{n+1} \otimes \mathbb{I}_n) W_p^\dagger (G_p + G_p^\dagger) W_p (\mathbb{I}_n \otimes |0\rangle_{n+1}) \tag{70}$$

Then, to prove the last theorem 6.1.1, we have shown that:

$$W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+1} |k\rangle_n = -2Re((-i)^p \psi_k) |0\rangle_n |0\rangle_1 |k\rangle_n \tag{71}$$

Here, based on the demonstration of the original property at [13], we infer that:

$$\begin{aligned} -W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle_{n+1} |k\rangle_n \langle k|_n \langle 0|_{n+1}) W_p^\dagger (G_p + G_p^\dagger) W_p = \\ 2 \sum_{i \neq j} Re((-i)^p \frac{A_{ij}}{2}) |i\rangle \langle j|_n \otimes |0\rangle \langle 0|_{n+1} + 2 \sum_i Re((-i)^p (\frac{A_{ii}}{2} + \frac{1}{2})) |i\rangle \langle j|_n \otimes |0\rangle \langle 0|_{n+1} \end{aligned} \tag{72}$$

All the coefficient of an adjacency matrix are real, which leads to:

$$-W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n) W_p^\dagger (G_p + G_p^\dagger) W_p = 2 \sum_{i \neq j} \frac{A_{ij}}{2} |i\rangle \langle j|_n \otimes |0\rangle \langle 0|_{n+1} + 2 \sum_i (\frac{A_{ii}}{2} + \frac{1}{2}) |i\rangle \langle i|_n \otimes |0\rangle \langle 0|_{n+1} \tag{73}$$

After that, one subtract the identity $\mathbb{I}_{n+2}$ to complete the calculation of the equation 68 and we get:

$$U^{(p)} (2 |0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n - \mathbb{I}_{2n+2}) U^{\dagger(p)} = \sum_{i,j} A_{ij} |i\rangle \langle j|_n \otimes |0\rangle \langle 0|_{n+1} \tag{74}$$

Then, looking at the equation 70, we see that we need to project onto $\mathbb{I}_n \otimes |0\rangle_{n+1}$ to get:

$$(\mathbb{I}_n \otimes \langle 0|_{n+1}) U^{(p)} (2 |0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n - \mathbb{I}_{2n+2}) U^{\dagger(p)} (\mathbb{I}_n \otimes |0\rangle_{n+1}) = \sum_{i,j} A_{ij} |i\rangle \langle j| \tag{75}$$

To summarize, we have:

$$(\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U^{(p)} (2 |0\rangle \langle 0|_{n+2} \otimes \mathbb{I}_n - \mathbb{I}_{2n+2}) U^{\dagger(p)} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n) = \sum_{ij} A_{ij} |i\rangle \langle j|_n \tag{76}$$

Which prove that we can block encode a non-diagonal matrix using this technique.

This technique could be useful for the case studied on Quandela's paper, since, now there is not apparent limitation on the probability boosting technique. One could multiply a row of an adjacency matrix without taking care of not having a singular value smaller than one. This would considerably boost the probability of getting the permanent of the matrix in output. Moreover, one just needs to implement two times the circuit needed for the theorem(6.1.1), to create U and $U^\dagger$ and also to implement a reflection around $|0\rangle_{n+2}$, which is done with depth $O(n)$. Thus, the depth of circuit would be $O(3n) = O(n)$.

Note that if we block encode diagonal matrix using 6.1.1, then we could perform any polynomials to its eigenvalues and so approximate any non-linear function. This opens a way to perform non linear calculations using linear optics circuit.

References

[1] Scott Aaronson and Alex Arkhipov. The computational complexity of linear optics, 2010.

[2] Guang Hao Low Nathan Wiebe András Gilyén, Yuan Su. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics, 2018.

[3] Simon Apers. Lecture 4: Quantum chemistry and simulation. 2023.

[4] Aleksandar Donev. SVD. https://cims.nyu.edu/~donev/Teaching/NMI-Fall2014/Lecture5.handout.pdf, 2024.

[5] M. Kpa et al. Simulation of charge noise affecting a quantum dot in a si/sige structure, 2023.

[6] N. Somaschi et al. Near-optimal single-photon sources in the solid state, 2016.

[7] Nicolas Maring et al. A general-purpose single-photon-based quantum computing platform, 2023.

[8] Juan Carlos Garcia-Escartin, Vicent Gimeno, and Julio José Moyano-Fernández. Method to determine which quantum operations can be realized with linear optics with a constructive implementation recipe. *Physical Review A*, 100(2), August 2019.

[9] Maxwell D. Radin Jérôme F. Gonthier and Corneliu Buda. Measurements as a roadblock to near-term practical quantum advantage in chemistry: Resource analysis, 2022.

[10] Eliott Rambeau Rodrigo Martinez. k-densest subgraph problem, added lines. https://github.com/Eliott-Rambeau/M2-project/blob/main/M2-project.ipynb, 2024.

[11] Rawad Mezher, Ana Filipa Carvalho, and Shane Mansfield. Solving graph problems with single-photons and linear optics, 2023.

[12] Hannes Moser. Exact algorithms for generalizations of vertex cover. *Institut für Informatik, Friedrich-Schiller-Universität Jena*, 12, 2005.

[13] Arthur G. Rattew1 and Patrick Rebentrost. Non-linear transformations of quantum amplitudes: Exponential improvement, generalization, and applications, 2023.

[14] Michael Reck and Anton Zeilinger. Experimental realization of any discrete unitary operator, 1994.

[15] Michael Reck, Anton Zeilinger, Herbert J. Bernstein, and Philip Bertani. Experimental realization of any discrete unitary operator. *Phys. Rev. Lett.*, 73:58–61, Jul 1994.

[16] Kenneth R. Rebman Russel Merris and William Watkins. Permanental polynomials of graphs, 1981.

[17] J. J. Schäffer. On unitary dilations of contractions. *Proceedings of the American Mathematical Society*, 6(2):322–322, 1955.

7 Appendix A

In order to prove this theorem, one firstly needs to defines several matrices:

Rotation matrix

$$R \coloneqq (\mathscr{I}_{n+1} - 2\left|0\right\rangle\left\langle 0\right|_{n+1}) \otimes \mathscr{I}_n \tag{77}$$

Control unitary

$$U_C \coloneqq (U \otimes \left|0\right\rangle\left\langle 0\right|_1 + \mathbb{I}_n \otimes \left|1\right\rangle\left\langle 1\right|_1) \otimes \mathbb{I}_n \tag{78}$$

Controlled-copy circuit

$$C \coloneqq \mathbb{I} \otimes \left|0\right\rangle\left\langle 0\right| \otimes \mathbb{I} + \sum_k \sum_j \left|j \oplus k\right\rangle\left\langle j\right| \otimes \left|1\right\rangle\left\langle 1\right| \otimes \left|k\right\rangle\left\langle k\right| \tag{79}$$

W operator

$$W_p \coloneqq HS^pCU_CH \tag{80}$$

Let $p \in \{0, 1\}$, then,

G operator

$$G_p \coloneqq W_pRW_p^\dagger Z \tag{81}$$

And finally, the **Total unitary**

$$\begin{aligned} U^{(p)} &\coloneqq (XZX \otimes \mathbb{I}_{2n+1})(H \otimes W_p^\dagger) \\ &(\left|0\right\rangle\left\langle 0\right|_1 \otimes G_p + \left|1\right\rangle\left\langle 1\right| \otimes G_p^\dagger)(H \otimes W_p) \end{aligned} \tag{82}$$

which is a (1,2n+1,0) block encoding of

$$\begin{aligned} A^{(p)} &= diag(Re(\psi_1), ..., Re(\psi_n)) \text{ if p=0} \\ A^{(p)} &= diag(Im(\psi_1), ..., Im(\psi_n)) \text{ if p=1} \end{aligned} \tag{83}$$

To prove this statement, one has to show that:

$$\|A^{(p)} - (\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U^{(p)} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n)\| = 0 \tag{84}$$

First of all, recall that $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$, which leads to:

$$\begin{aligned} U^{(p)} &= (XZX|+\rangle\langle+|) \otimes (W_p^\dagger G_p W_p) \\ &\quad + (XZX|-\rangle\langle-|) \otimes (W_p^\dagger G_p^\dagger W_p) \end{aligned} \tag{85}$$

$$\begin{aligned} \iff U^{(p)} &= -(|-\rangle\langle+|) \otimes (W_p^\dagger G_p W_p) + \\ &\quad (|+\rangle\langle-|) \otimes (W_p^\dagger G_p^\dagger W_p) \end{aligned} \tag{86}$$

Then,

$$\begin{aligned} &(\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U^{(p)} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n) = \\ &-\frac{1}{2} (\langle 0|^{\otimes n+1} \otimes \mathbb{I}_n) W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle^{\otimes n+1} \otimes \mathbb{I}_n) \end{aligned} \tag{87}$$

After that, we want to find the elements of this matrix. Which are:

$$-\frac{1}{2} \langle j|_n (\langle 0|^{\otimes n+1} \otimes \mathbb{I}_n) W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle^{\otimes n+1} \otimes \mathbb{I}_n) |k\rangle_n \tag{88}$$

This reduces to compute:

$$W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+1} |k\rangle_n \tag{89}$$

Let's start with the second term:

$$W_p^\dagger G^\dagger W_p |0\rangle^{\otimes n+1} |k\rangle_n = W_p^\dagger Z W_p R W_p^\dagger W_p |0\rangle^{\otimes n+1} |k\rangle_n \tag{90}$$

$$\iff W_p^\dagger G^\dagger W_p |0\rangle^{\otimes n+1} |k\rangle_n = W_p^\dagger Z W_p R |0\rangle^{\otimes n+1} |k\rangle_n \tag{91}$$

$$\iff W_p^\dagger G^\dagger W_p |0\rangle^{\otimes n+1} |k\rangle_n = -W_p^\dagger Z W_p |0\rangle^{\otimes n+1} |k\rangle_n \tag{92}$$

Then, one can define:

$$|\phi_p\rangle \coloneqq W_p |0\rangle^{\otimes n+1} |k\rangle_n \tag{93}$$

And we find:

$$W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+1} |k\rangle_n = W_p^\dagger (W_p R W_p^\dagger Z - Z) W_p |0\rangle^{\otimes n+1} |k\rangle_n \tag{94}$$

$$\iff W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+1} |k\rangle_n = W_p^\dagger (W_p R W_p^\dagger - \mathbb{I}_{2n+1}) Z |\phi_p\rangle \tag{95}$$

After that, one can compute the term:

$$\begin{aligned} W_p R W_p^\dagger &= W_p (\mathbb{I}_{n+1} - 2|0\rangle\langle 0|_{n+1}) \otimes \mathbb{I}_n W_p^\dagger \\ &= \mathbb{I}_{2n+1} - 2W_p (|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n) W_p^\dagger \end{aligned} \tag{96}$$

Which leads to:

$$W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+1} |k\rangle_n = -2(|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n) W_p^\dagger Z |\phi_p\rangle \tag{97}$$

To simplify this equation, note that:

$$Z|\phi_p\rangle = \frac{1}{2}((|\psi\rangle_n + i^p |k\rangle_n)|0\rangle_1 - (|\psi\rangle_n - i^p |k\rangle_n)|1\rangle_1)|k\rangle_n \tag{98}$$

Then, we have $W_p^\dagger = H U_c^\dagger C^\dagger (S^p)^\dagger H$. Applying term by term, one gets:

$$H\frac{1}{2}((|\psi\rangle_n + i^p |k\rangle_n)|0\rangle_1 - (|\psi\rangle_n - i^p |k\rangle_n)|1\rangle_1)|k\rangle_n = \frac{1}{2}((|\psi\rangle_n + i^p |k\rangle_n)|+\rangle_1 - (|\psi\rangle_n - i^p |k\rangle_n)|-\rangle_1)|k\rangle_n \tag{99}$$

$$
\begin{aligned}
(S^p)^\dagger \frac{1}{2}((|\psi\rangle_n + i^p |k\rangle_n)|+\rangle_1 - (|\psi\rangle_n - i^p |k\rangle_n)|-\rangle_1)|k\rangle_n = \\
\frac{1}{\sqrt{2}}(i^p |k\rangle_n |0\rangle_1 + (-i)^p |\psi\rangle_n |1\rangle_1)|k\rangle_n = \\
\frac{i^p}{\sqrt{2}}(|k\rangle_n |0\rangle_1 + (-1)^p |\psi\rangle_n |1\rangle_1)|k\rangle_n
\end{aligned} \tag{100}
$$

And, note that $C^\dagger = \mathbb{I} \otimes |0\rangle\langle 0| \otimes \mathbb{I} + \sum_k \sum_j |j\rangle\langle j \oplus k| \otimes |1\rangle\langle 1| \otimes |k\rangle\langle k|$, we get

$$
C^\dagger \frac{i^p}{\sqrt{2}}(|k\rangle_n |0\rangle_1 + (-1)^p |\psi\rangle_n |1\rangle_1)|k\rangle_n = \frac{i^p}{\sqrt{2}}(|k\rangle_n |0\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |1\rangle_1)|k\rangle_n \tag{101}
$$

After that, we have $U_c^\dagger = (U \otimes |0\rangle\langle 0|_1 + \mathbb{I}_n \otimes |1\rangle\langle 1|_1) \otimes \mathbb{I}_n$, which leads to:

$$
U^\dagger \frac{i^p}{\sqrt{2}}(|k\rangle_n |0\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |1\rangle_1)|k\rangle_n = \frac{i^p}{\sqrt{2}}(U^\dagger |k\rangle_n |0\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |1\rangle_1)|k\rangle_n \tag{102}
$$

And:

$$
H\frac{i^p}{\sqrt{2}}(U^\dagger |k\rangle_n |0\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |1\rangle_1)|k\rangle_n = \frac{i^p}{\sqrt{2}}(U^\dagger |k\rangle_n |+\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |-\rangle_1)|k\rangle_n \tag{103}
$$

Finally, we apply $|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n$ to get:

$$
\begin{aligned}
(|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n) W_p^\dagger Z |\phi_p\rangle \\
= \frac{i^p}{\sqrt{2}}(|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n)(U^\dagger |k\rangle_n |+\rangle_1 + (-1)^p \sum_{j=0}^{N-1} \psi_{j\oplus k} |j\rangle_n |-\rangle_1)|k\rangle_n \\
= \frac{i^p}{2}(|0\rangle_n \langle\psi|k\rangle |0\rangle + (-1)^p \psi_k |0\rangle_n |0\rangle_1)|k\rangle_n \\
= \frac{i^p}{2}(\psi_k^* + (-1)^p \psi_k)|0\rangle_n |0\rangle_1 |k\rangle_n
\end{aligned} \tag{104}
$$

And so:

$$
\begin{aligned}
W_p^\dagger (G_p + G_p^\dagger) W_p |0\rangle^{\otimes n+2} |k\rangle_n \\
= -2(|0\rangle\langle 0|_{n+1} \otimes \mathbb{I}_n) W_p^\dagger Z |\phi_p\rangle \\
= -i^p(\psi_k^* + (-1)^p \psi_k)|0\rangle_n |0\rangle_1 |k\rangle_n \\
= -2Re((-i)^p \psi_k)|0\rangle_n |0\rangle_1 |k\rangle_n
\end{aligned} \tag{105}
$$

We finally have:

$$
\begin{aligned}
(\langle 0|^{\otimes n+2} \otimes \mathbb{I}_n) U^{(p)} (|0\rangle^{\otimes n+2} \otimes \mathbb{I}_n) = \\
-\frac{1}{2}(\langle 0|^{\otimes n+1} \otimes \langle j|_n) W_p^\dagger (G_p + G_p^\dagger) W_p (|0\rangle^{\otimes n+1} \otimes |k\rangle_n) = \\
(\langle 0|^{\otimes n+1} \otimes \langle j|_n) Re((-i)^p \psi_k)(|0\rangle_n |0\rangle_1 |k\rangle_n) = \\
\begin{cases} 0 \text{ if } j \neq k \\ Re((-i)^p \psi_k) \text{ otherwise} \end{cases}
\end{aligned} \tag{106}
$$

Thus, the target matrix is diagonal, and its diagonal elements have the form: $Re((-i)^p \psi_k)$, which proves the equation 84, and so the fact that one can block-encode a diagonal matrix.
Then, one can use the linear combination theorem for Block encoding matrix in [2] to prove that we can block encode the matrix $A^{(p)} = A^{(r)} + iA^{(i)}$.
The only thing left is to prove the complexity part of the theorem. This procedure is clear, since one just needs to look at the definitions of the different matrices and count the number of query to U and $U^\dagger$. □